

3.c. Multi – Level linked list
–Insert At Position
–Space Complexity and Algorithm

Program:

```
void insertAtPosition()
{
    int data, pos;
    cout << "Enter data: ";
    cin >> data;
    cout << "Enter position (1-based): ";
    cin >> pos;

    Node *newNode = createNode(data);

    if (pos == 1)
    {
        newNode->next = head;
        head = newNode;
        return;
    }

    Node *temp = head;
    for (int i = 1; i < pos - 1 && temp != nullptr; i++)
    {
        temp = temp->next;
    }

    if (temp == nullptr)
    {
        cout << "Position out of range." << endl;
        free(newNode);
        return;
    }
}
```

```

else
{
    newNode->next = temp->next;
    temp->next = newNode;
    cout << "Inserted at position " << pos << endl;
}
}
.....
int main()
{
    int choice;
    while (true)
    {
        cout << "3. Insert at Position (Main)" << endl;

        cin >> choice;
        switch (choice)
        {
            case 3:
                insertAtPosition();
                break;
            default:
                cout << "Invalid choice" << endl;
        }
    }
    return 0;
}

```

Space Complexity

A. Input Space

A. Parameter – Only Definition

Under a strict, parameter – only definition, input space is evaluated solely based on the explicit arguments passed into the function signature.

- *void insertAtPosition(). It accepts **no explicit parameters**.*
- *The data (data, pos) is pulled from the standard input stream (cin) at runtime rather than being passed as arguments.*
- *The function relies on a globally or class – scoped head pointer to access the linked list.*

*Under the strictest parameter – only academic definition, the input space footprint of this specific function call is **O(1)**, as it only requires the memory of the implicit head pointer to begin its operation.*

B. Full Data Definition (Holistic Approach)

The holistic approach evaluates the total memory footprint of the underlying data structure being manipulated. Algorithms rarely operate in a vacuum, and understanding the total memory pressure is vital.

- *The function operates on a linked list consisting of N existing nodes.*
- *Each of the N nodes occupies memory (e. g., 16 bytes each, considering the data, pointer, and structural padding).*
- *The total memory actively held by the data structure traversing the heap is proportional to the number of elements.*

*Under the holistic definition, the input space of the linked list is **O(N)**, where N is the number of nodes currently in the list.*

B. Auxiliary Space

Auxiliary space means:

Extra temporary memory used by the algorithm excluding the input itself.

Now analyze carefully.

Local Variables:

```
int data, pos;  
Node *newNode;  
Node *temp;  
int i;
```

Inside function of InsertAtPosition:

```
int data;  
Node *newNode;
```

Also in createNode(int data) function:

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

Here ,both the newNode contains dynamically memory addresses, where as ,dynamically allocated memory :

```
(Node *)malloc(sizeof(Node));
```

itself is input space. But not:

```
Node *newNode;
```

*Hence Node * newNode; $\rightarrow O(1)$ auxiliary space.*

```
for (int i = 1; i < pos - 1 && temp != nullptr; i++)  
{  
    temp = temp->next;  
}
```

The loop ITERATES over many nodes,

but does NOT STORE many nodes.

This is the key distinction.

The SAME pointer variable is reused repeatedly.

No new pointers are created.

No array is created.

No recursion stack grows.

No extra linked list is formed.

Thus memory usage stays constant.

Thus Auxiliary Space is Constant $\rightarrow O(1)$

Total Space Complexity

Total Space Complexity = Auxiliary Space + Input Space

Based on the definition of input space you use, the total space complexity changes.

- **Common Interpretation (Parameter-only input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(1)}_{\substack{\downarrow \\ \text{Input}}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(n)}_{\substack{\downarrow \\ \text{Input}}} = O(n).$$

1) If Asked “Total Space Complexity” then natural answer is
: $O(n)$ [*Full Data + Auxiliary*]

2) *If not asked total space complexity then:*

In most academic and interview contexts, when asked for the space complexity of a function, the **auxiliary space complexity ($O(1)$)** is the expected answer.

In most academic DSA contexts and coding interviews, when someone simply asks:

“What is the space complexity?”

they usually mean:

Auxiliary Space Complexity

unless they explicitly say:

- total space complexity
- overall memory usage
- data structure space
- input space
- storage complexity

So for your insertion function, the expected answer is usually:

$O(1)$

because only constant extra working memory is used.

Algorithm:

MultiINSPOS():

1. *Declare two integer type variable data, pos.*
 2. *Read data.*
 3. *Read pos.*
 4. *SET NEWPTR := CALL CreateNode(data)*
-

Translating CreateNode(data)

*((Node *)malloc(sizeof(Node)) is availability of space for insertion of Data, hence, lets name it AVAIL.))*

CreateNode(data):

1. *Allocate New Node*

SET NewTempPTR := AVAIL.

2. *Memory Allocated ?:*

If NewTempPTR = NULL, then:

Write: " Error: Memory allocation failed."

Return and EXIT.

[End of If structure.]

3. *Copy Data:*

Set INFO[NewTempPTR] := data

4. *Set Next section of Allocated Memory:*

Set Link[NewTempPTR] := NULL

5. *Set Child section of Allocated Memory:*

Set childLink[NewTempPTR] := NULL

6. *Return Stored Address in NewTempPTR*

return NewTempPTR

5. *[ENTER NODE in 1st Position ?]*

If pos = 1 , then:

SET LINK[NEWPTR] := START

SET START := NEWPTR

RETURN

[End of If structure.]

6. *[ENTER NODE > 1st Position?]*

SET TEMPPTR := START $\left[\begin{array}{l} \text{Set TemporaryPointer} \\ \text{to 1st Node which will} \\ \text{help to traverse the loop.} \end{array} \right]$

7. *[LOOP TRAVERSAL]*

FOR i = 1; i < POS – 1 AND TEMP ≠ NULLPTR; DO:

SET TEMPPTR := LINK[TEMPPTR]

SET i := i + 1

8. *[If Out of Bound ?]*

If TEMPPTR = NULL, THEN:

Write: ``POSITION OUT OF RANGE``

FREE (NEWPTR)

RETURN

[End of If structure.]

ELSE

SET LINK[NEWPTR] := LINK[TEMPPTR]

*[LINKING NEWNODE with the
previous node after which
newNode is to be placed
in LINK LIST]*

SET LINK[TEMPPTR] := NEWPTR

*[LINKING NEWNODE with the
NEXT NODE where this
NEW NODE is to be placed
in LINK LIST]*

Write : ``Inserted at position``, pos.

9. END OF ALGORITHM
